RELIABLE DATA TRANSFER

CIS*3210
Fall 2026

Sajid Marhon

Based on notes for Computer Networking: A Top-Down Approach, 9th ed., by J. Kurose and K. Ross

Learning Outcomes



By the end of this lesson, students will be able to:

- Describe the role of **reliable data transfer** in networking
- Interpret sender/receiver **FSMs (Finite-State Machines)** for RDT protocols
- Explain how **checksums, ACK/NAK, sequence numbers, and timers** ensure reliability
- Distinguish between **stop-and-wait** and **pipelined protocols**
- Relate RDT concepts to **TCP**

Outline



- Principles of reliable data transfer
- RDT interfaces and finite-state machines
- Building RDT protocols: rdt1.0 to rdt3.0
- Stop-and-wait operation and performance
- Pipelining and sliding windows
- Go-Back-N (GBN)
- Selective Repeat (SR)
- Comparison of reliable data transfer protocols

Reliable Data Transfer



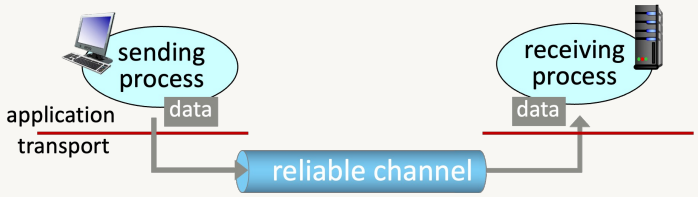
- **Reliable Data Transfer (RDT) Problem**
  - A core issue in networking, present at the **transport, link, and application layers**
  - One of the **most fundamental challenges** in network design
  - TCP is a prime example of an RDT protocol
- **Service abstraction**
  - Provides an illusion of a **reliable channel**:
    - No bit corruption
    - No packet loss
    - In-order delivery
  - This is what TCP offers to Internet applications

This doesn't mean that the network is reliable

Reliable Data Transfer: Service Abstraction



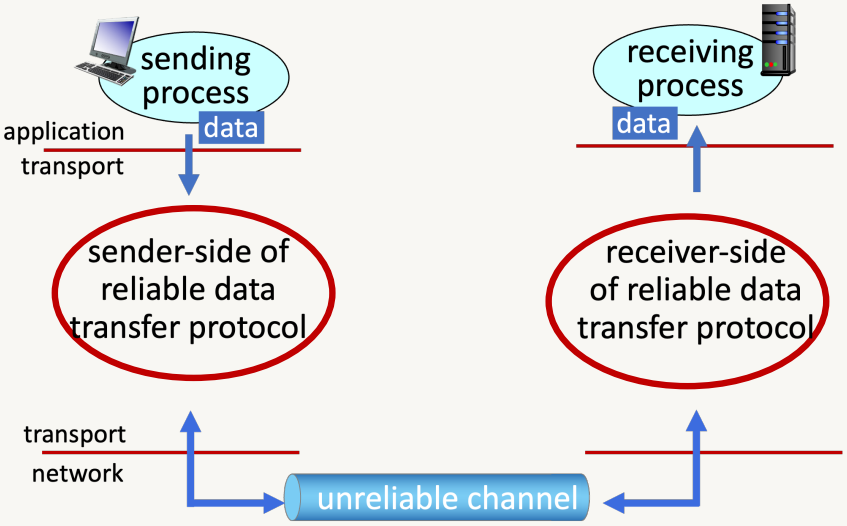
Data flows in one direction through the reliable channel, from sender to receiver



reliable service *abstraction*

Implementing Reliable Data Transfer





reliable service *implementation*

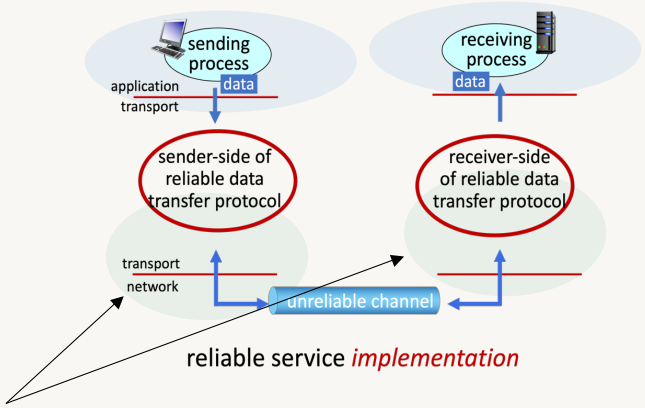
Communication over the unreliable channel is two-way: the sender and receiver exchange messages to implement one-way reliable data transfer



Discussion Question:

The receiving application gets every message once, in the correct order, but a Wireshark packet capture shows corrupted packets and repeated transmissions. Does the capture contradict the claim that the application received reliable service?

No. Reliable service describes what the upper layer receives, not every packet sent through the channel. The protocol can detect corrupted packets, retransmit data, and prevent duplicate delivery while hiding these channel problems from the application.



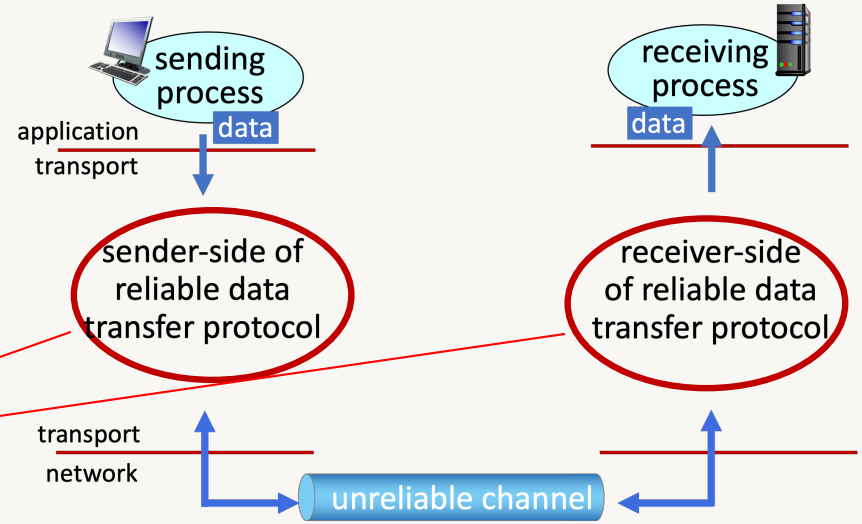
Application view: Correct data, delivered once and in order

Wireshark view: Packet traffic may include corruption and retransmissions

How Channel Errors Shape RDT Design



The design complexity of an RDT protocol is heavily influenced by the type of channel errors (loss, corruption, reordering).

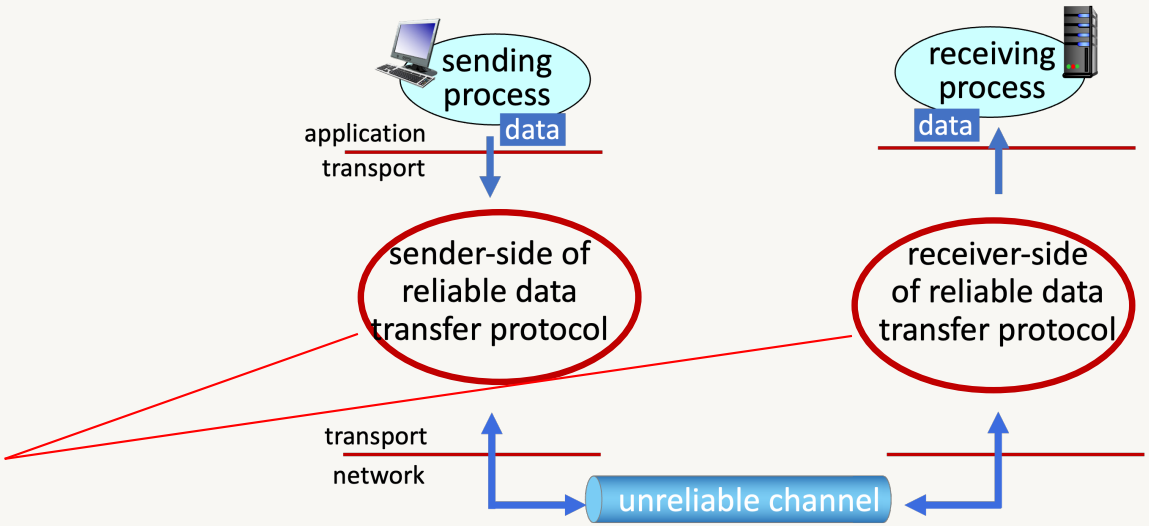


reliable service *implementation*

Why RDT Needs Feedback



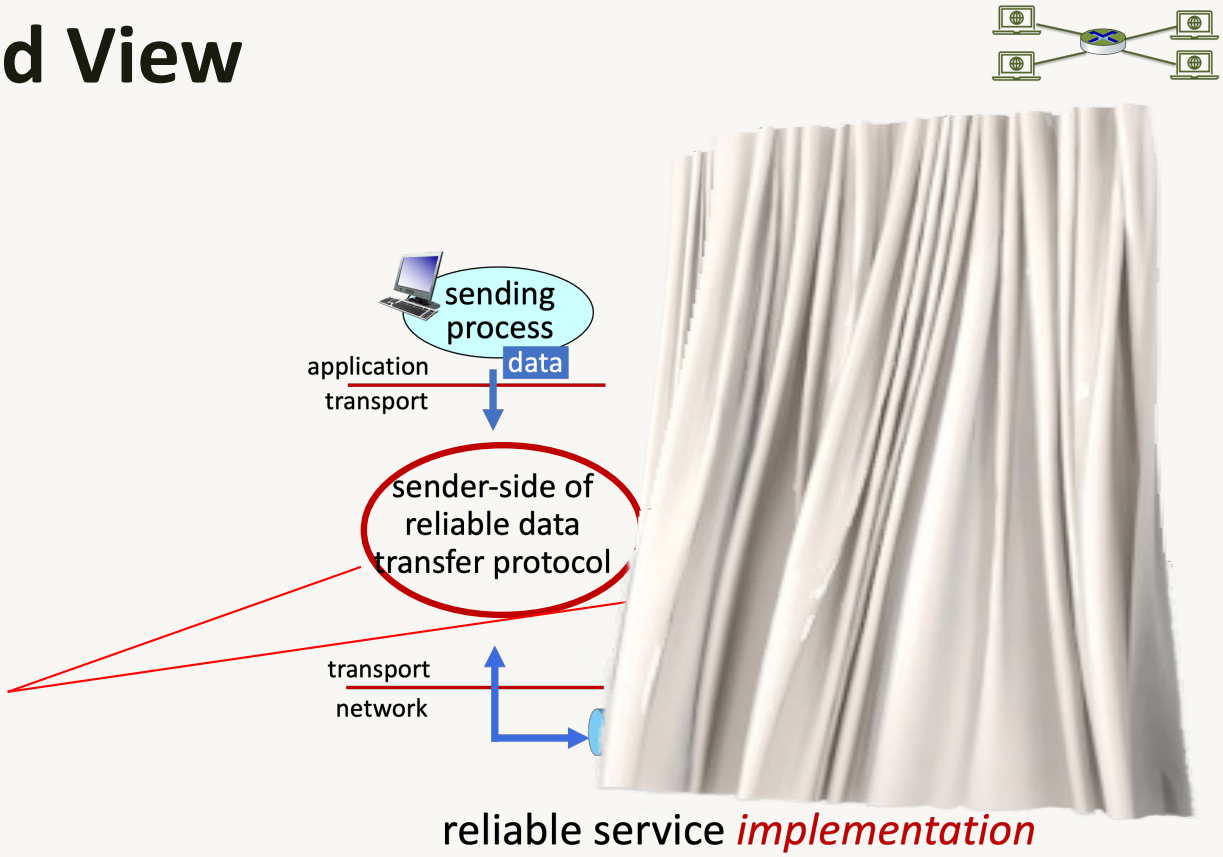
Sender and receiver are unaware of each other's state (e.g., message received) unless it's explicitly communicated.



reliable service *implementation*

The Sender's Limited View

Sender and receiver are unaware of each other's state (e.g., message received) unless it's explicitly communicated.





Discussion Question:

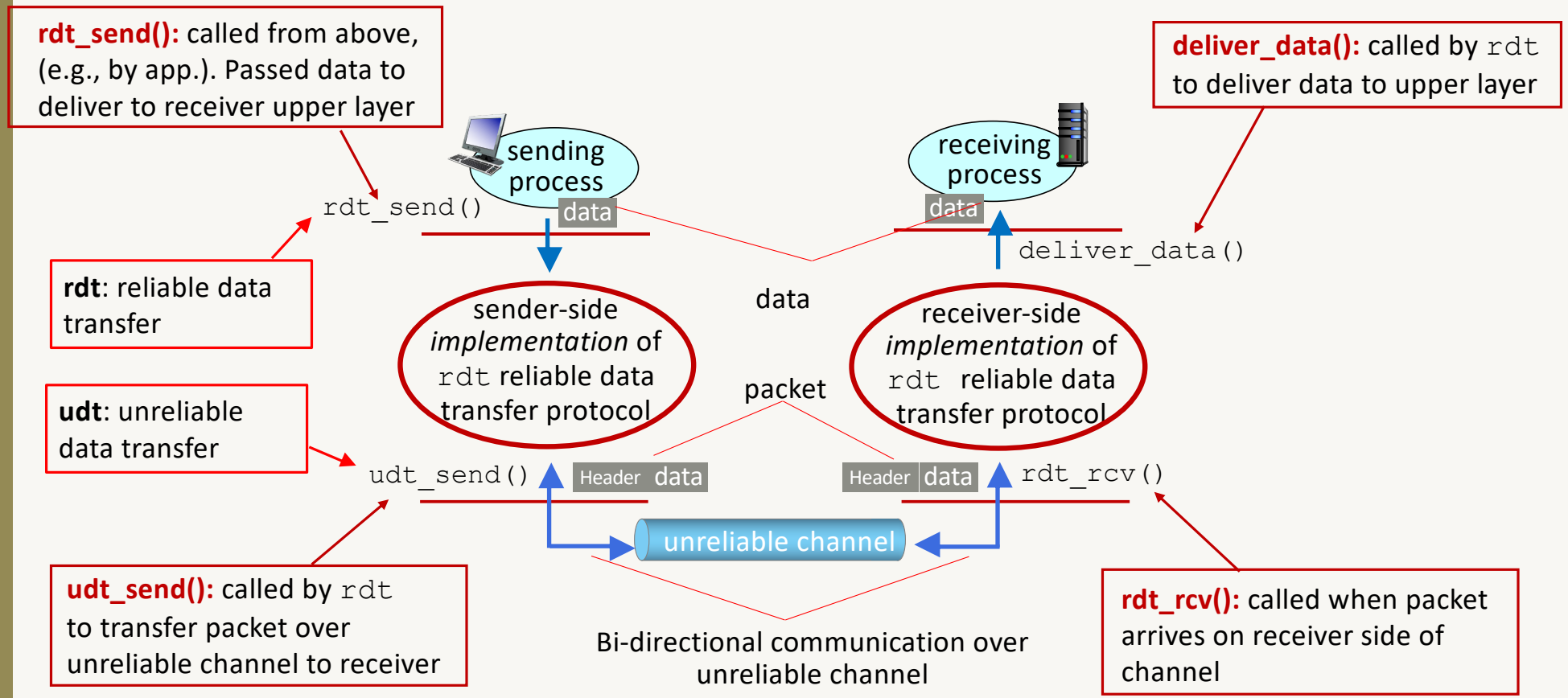
A sender has transmitted a packet and has received no reply. Give three different channel events that could produce this observation. Why would immediately declaring the data packet lost be unjustified?

The data packet could be lost, the receiver's reply could be lost, or the data or reply could still be delayed.

The sender observes only the absence of feedback. That observation does not identify which event occurred or prove that the receiver lacks the data.

RDT Protocol Interfaces

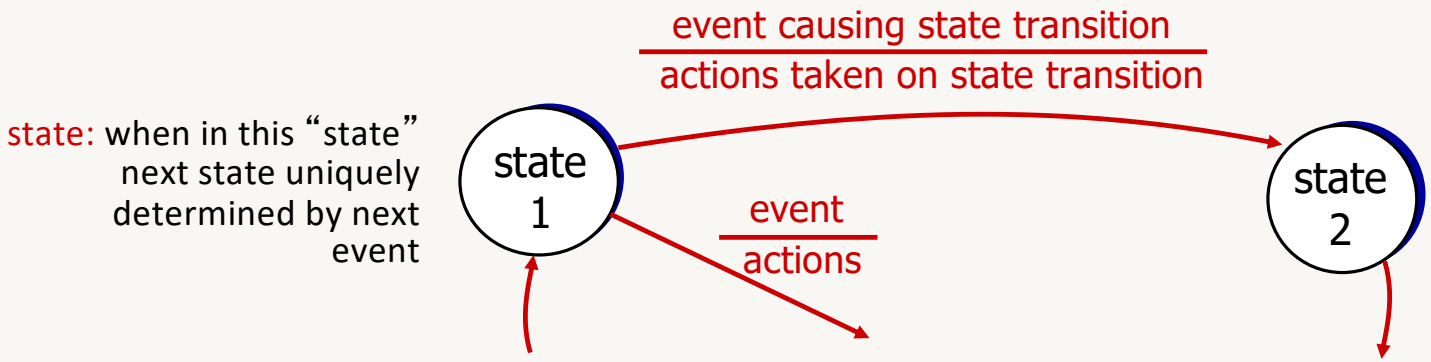




RDT Finite-State Machines



- We will build the sender and receiver of RDT step by step, focusing on one-way data transfer, while control messages still move both ways
- **FSMs (finite-state machines)**:
  - Separate FSMs for sender and receiver
  - Each FSM has a finite set of states
  - Transitions may move between states or return to the same state



Building Reliable Data Transfer



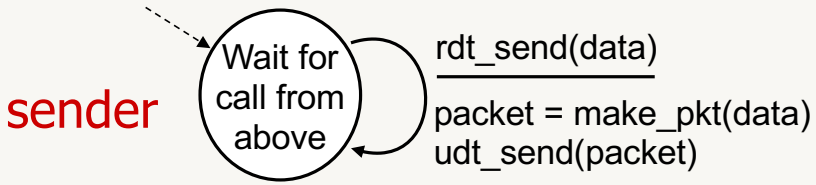
Teaching Models

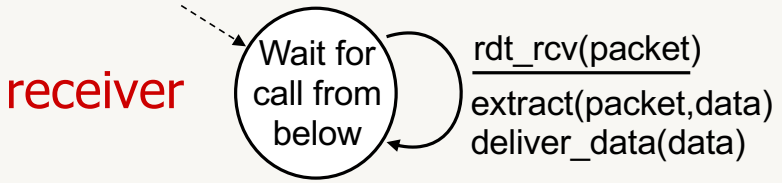
RDT / Protocol	Handles Bit Errors?	Handles Packet Loss?	Handles Corrupted ACK/NAK?	Handles Out-of-Order Packets?	Protocol Type
rdt1.0					
rdt2.0					
rdt2.1					
rdt2.2					
rdt3.0					
Go-Back-N (GBN)					
Selective Repeat (SR)					

rdt1.0: RDT Over a Reliable Channel



- **Underlying channel perfectly reliable**
  - no bit errors
  - no loss of packets
- **Independent FSMs for sender and receiver:**
  - Sender: pushes data to the channel
  - Receiver: retrieves data from the channel





- **Sender actions**:
  - Accepts data from upper layer (rdt_send)
  - Creates packet (make_pkt(data))
  - Sends packet to channel (udt_send(packet))
- **Receiver actions**:
  - Accepts packet from channel (rdt_rcv(packet))
  - Extracts data from packet (extract(packet, data))
  - Delivers data to upper layer (deliver_data(data))

rdt1.0: Summary



RDT / Protocol	Handles Bit Errors?	Handles Packet Loss?	Handles Corrupted ACK/NAK?	Handles Out-of-Order Packets?	Protocol Type
rdt1.0	Perfect channel → no errors	✗	N/A—no ACKs or NAKs used	✗	Simple transfer (no ACKs)



Discussion Question:

An rdt1.0 receiver is given the same correct packet twice because the channel unexpectedly duplicates it. Why can it not safely decide that the second packet is a duplicate merely by comparing the data values?

The application may legitimately send the same data twice. Equal data values do not distinguish a repeated transmission from a new message with identical contents.

rdt1.0 has no packet identity mechanism for making that distinction because duplication is outside its channel assumptions.

rdt2.0: Channel With Bit Errors



- More realistic model: packets may be received with **corrupted bits**
- Packets are still delivered **in order** and not lost
- How to recover from errors?

How do humans recover from "errors" during conversation?

- **Human analogy (dictation)**
  - Receiver says **"OK"** (ACK) for correct sentences
  - Says **"Repeat"** (NAK) for garbled sentences
  - Basis for **ARQ (Automatic Repeat reQuest)** protocols

rdt2.0: Channel With Bit Errors



- **ARQ protocol requirements**
  - **Error detection** → checksum bits detect corruption
  - How to recover from errors?
    - **acknowledgements (ACKs):** receiver explicitly tells sender that packet received OK
    - **negative acknowledgements (NAKs):** receiver explicitly tells sender that packet had errors
    - sender **retransmits** packet on receipt of NAK
- **Also known as Stop-and-Wait protocol**
- Sender:
  - Waits for data → creates packet with checksum → sends
  - Waits for ACK/NAK
  - Sends new data only after correct ACK is received
- Receiver:
  - On packet arrival, sends ACK if good, NAK if corrupted
- Limitation: sender cannot accept new data until current packet acknowledged

rdt2.0: FSM Specifications



Sender FSM (two states)

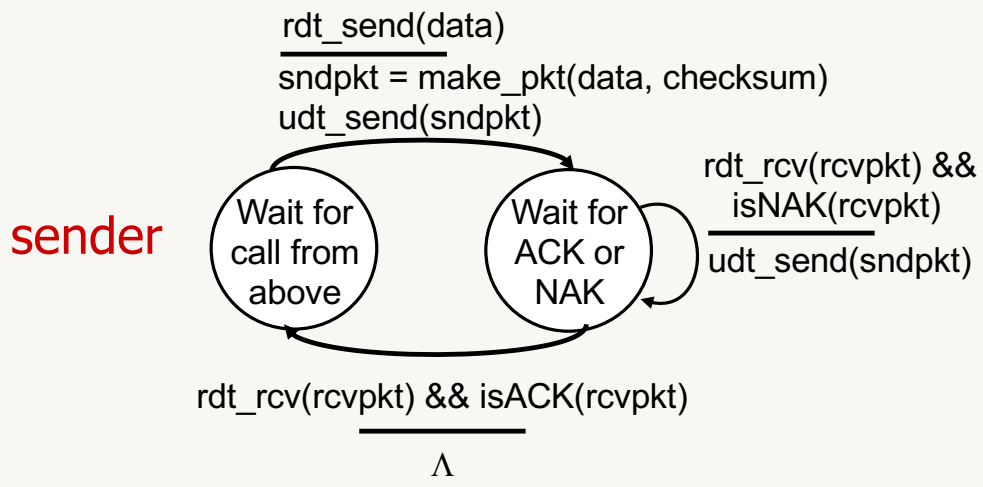
- **Waiting for data (left state):**
  - On rdt_send(data): create packet with checksum (sndpkt)
  - Send packet using udt_send(sndpkt)
- **Waiting for ACK/NAK (right state):**
  - On **ACK**: packet confirmed, return to **Waiting for data** (rdt_rcv(rcvpkt) && isACK(rcvpkt))
  - On **NAK**: retransmit last packet, **stay in this state** (rdt_rcv(rcvpkt) && isNAK(rcvpkt))
  - No new data accepted until a valid ACK is received → enforces stop-and-wait
  - This behaviour defines a stop-and-wait protocol

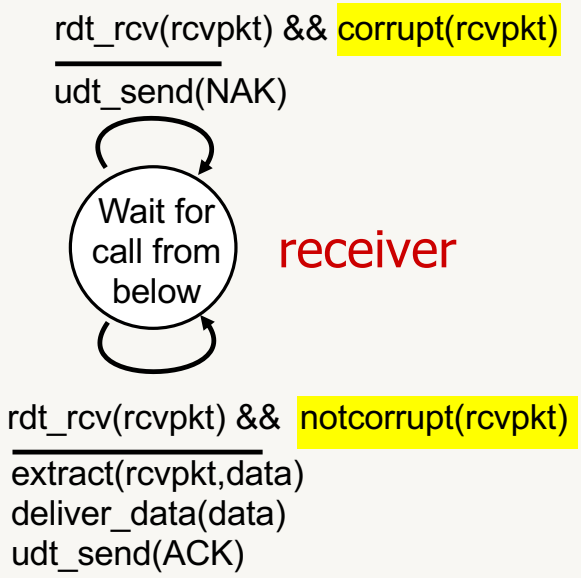
Receiver FSM (one state)

- **On packet arrival:**
  - If packet correct → send **ACK**
  - If packet corrupted → send **NAK**

rdt2.0: FSM Specifications



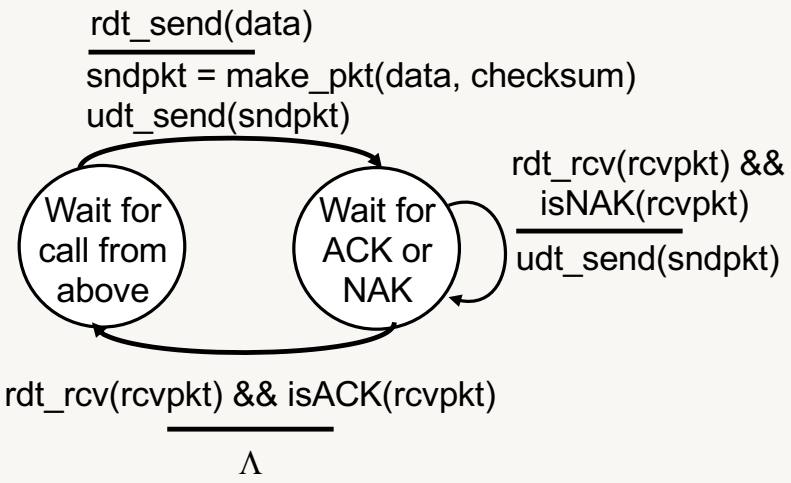




rdt2.0: FSM Specifications



sender

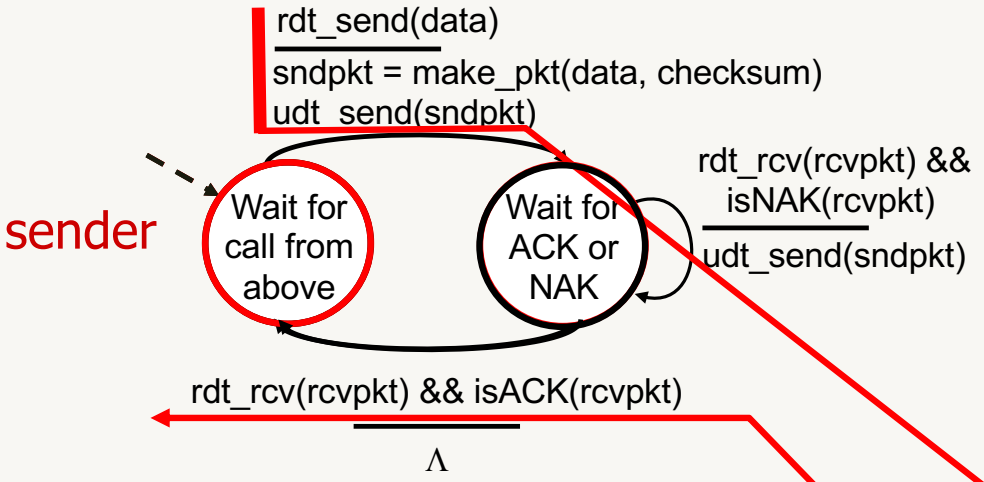


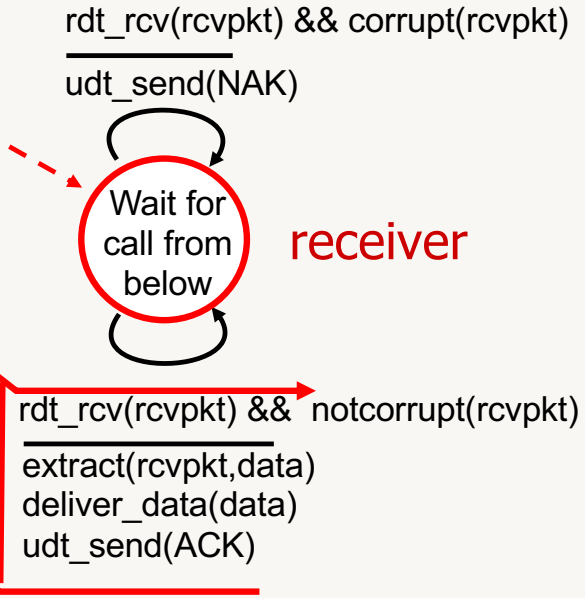
Note: The sender needs feedback from the receiver to know whether the packet was received correctly. This necessity is why protocols exist

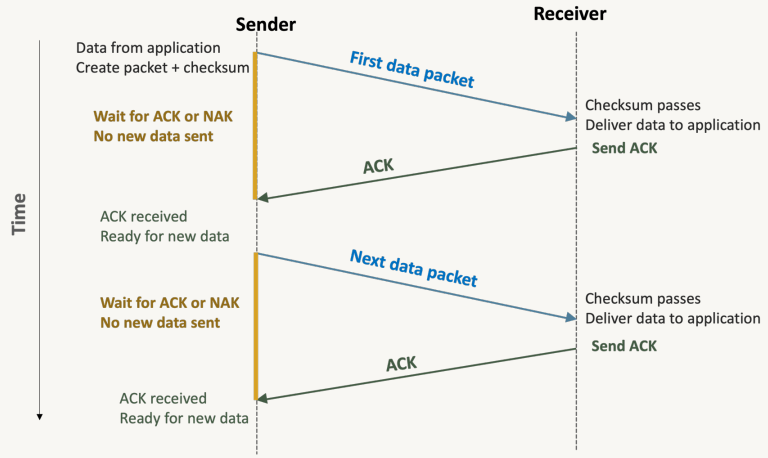


rdt2.0: Operation with No Errors



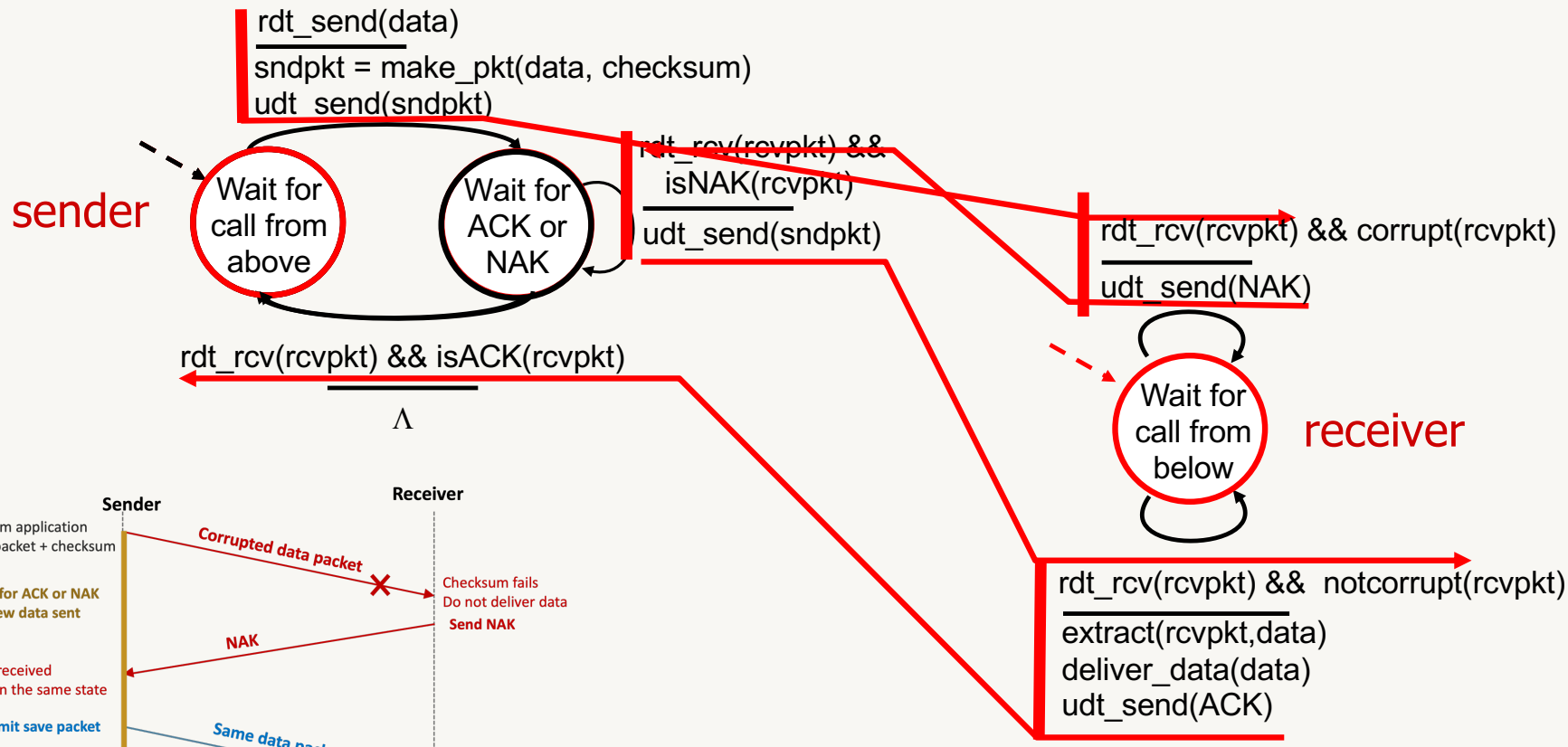


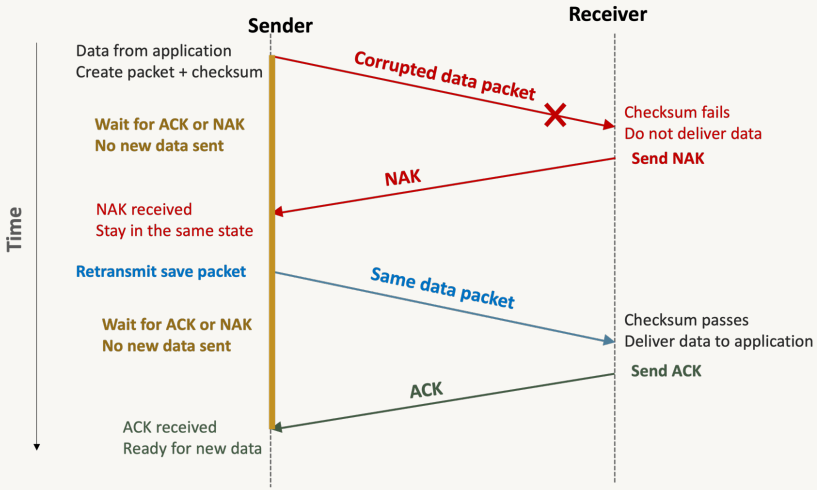




rdt2.0: Corrupted Packet Scenario





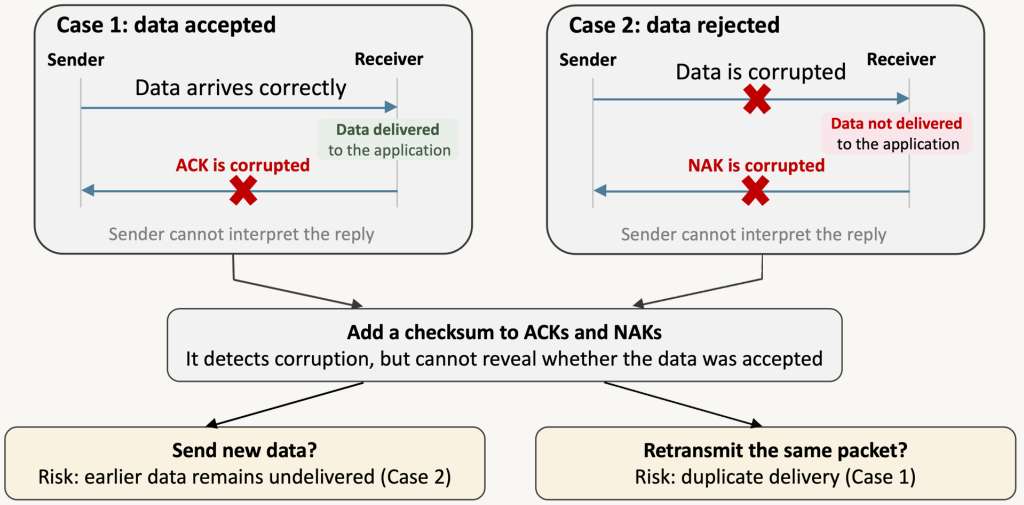


Flaw in rdt2.0



- **Flaw in protocol (rdt2.0)**
  - Doesn't handle the case where **ACK/NAK packets get corrupted**
  - Sender can't know if the last data packet was received correctly
- **Minimum fix**
  - Add **checksums** to ACK/NAK packets to detect errors
  - But detection alone doesn't solve the recovery problem

Two different receiver outcomes can look the same to the sender



Flaw in rdt2.0



Three possible solutions

- **Repeat request ("What did you say?")**
  - Sender asks receiver to resend its ACK/NAK
  - Risk: repeated corruption leads to confusion and endless loops
- **Stronger error correction**
  - Use an error-correcting code for ACK/NAK packets
  - Corrects errors within the code's correction capability; does not handle packet loss
- **Retransmit last data packet**
  - Sender resends when ACK/NAK is garbled
  - Problem: may cause **duplicate packets**, since receiver can't tell if it's new data or retransmission

rdt2.0: Summary



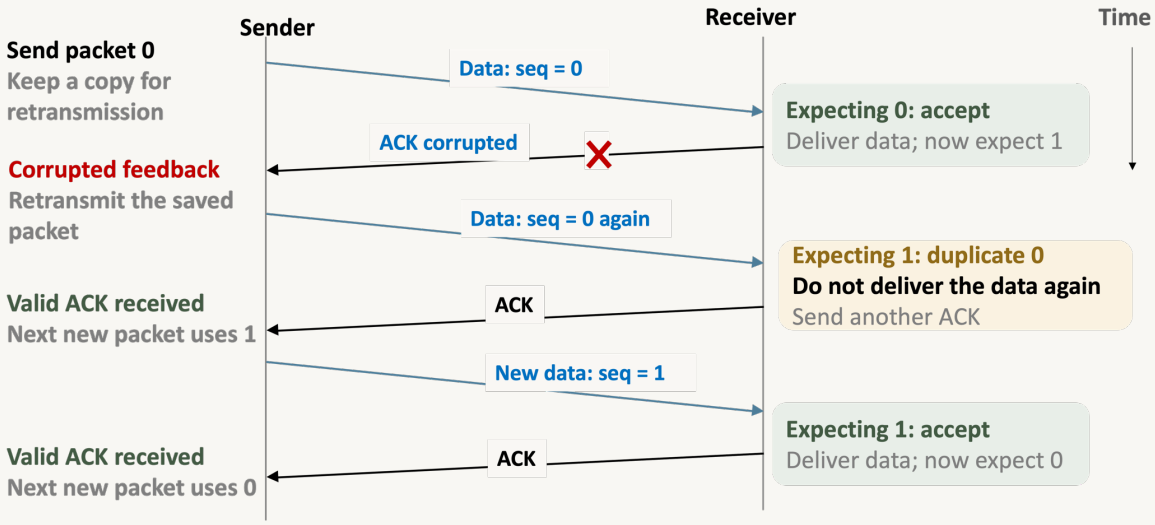
RDT / Protocol	Handles Bit Errors?	Handles Packet Loss?	Handles Corrupted ACK/NAK?	Handles Out-of-Order Packets?	Protocol Type
rdt1.0	Perfect channel → no errors	❌	N/A—no ACKs or NAKs used	❌	Simple transfer (no ACKs)
rdt2.0	✅	❌	❌ (does not handle corrupted ACK/NAK)	❌	Stop-and-Wait

rdt2.1: Preventing Duplicate Delivery



- Add a **sequence number** field to data packets
- Receiver checks sequence number to distinguish **new vs. duplicate** packets
- For stop-and-wait, a **1-bit sequence number (0/1)** is enough
- ACK/NAK packets don't need sequence numbers since they always refer to the last transmitted data packet

A corrupted ACK triggers a retransmission with the same sequence number



rdt2.1: Tracking Packets and Handling Duplicates



- **Updated protocol (rdt2.1)**
  - Sender and receiver FSMs now have **twice as many states**
  - Each state reflects whether the current/expected packet has **sequence number 0 or 1**
- **State behaviour**
  - States for **seq=0** mirror those for **seq=1**
  - The only difference is in how the **sequence number** is handled
- **Protocol behaviour**
  - Uses **ACKs (positive acknowledgements)** and **NAKs (negative acknowledgements)**
  - **Duplicate packet** → receiver sends ACK for the correctly received packet
  - **Corrupted packet** → receiver sends NAK

rdt2.1: Sender States and Error Recovery



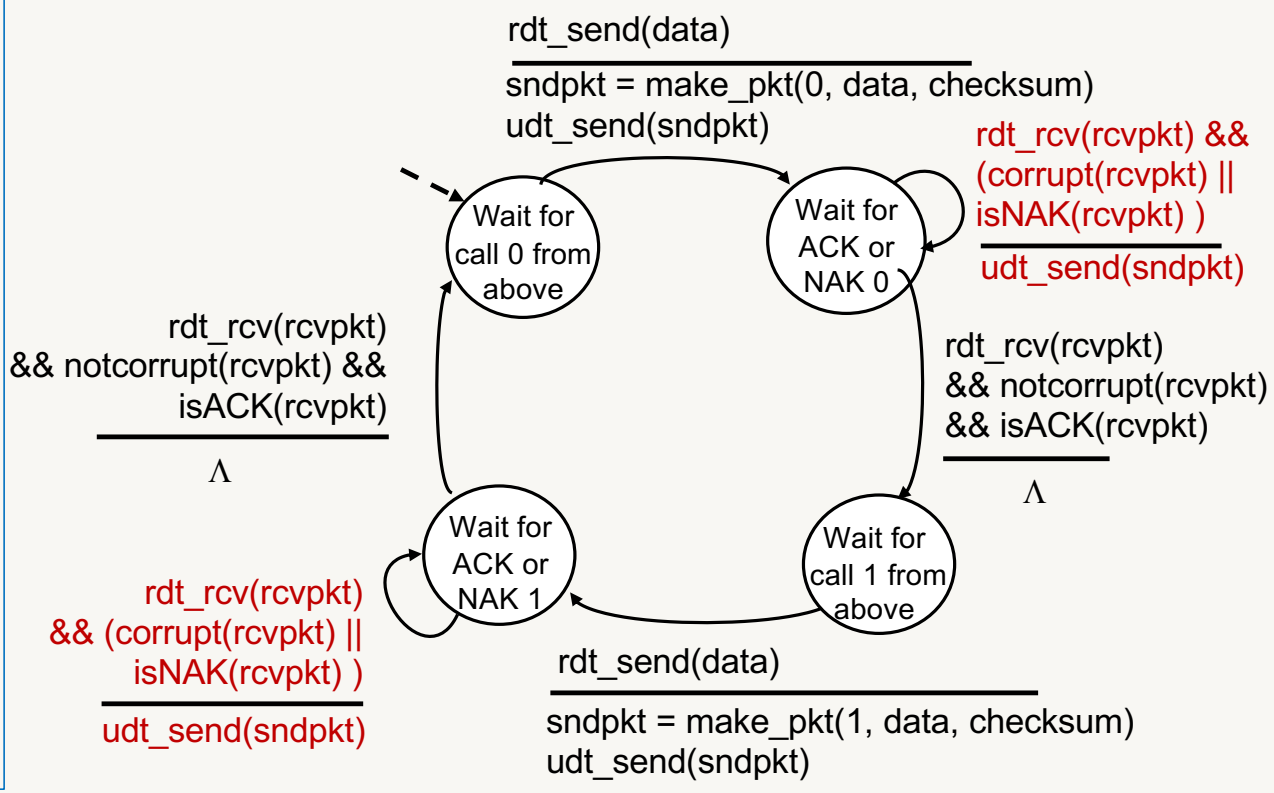
States

1. Wait for call 0 from above
- Sender is ready to send a new packet with sequence number **0**

2. Wait for ACK or NAK 0
- Sender has sent a seq=0 packet and is waiting for feedback

3. Wait for call 1 from above
- Sender is ready to send a new packet with sequence number **1**

4. Wait for ACK or NAK 1
- Sender has sent a seq=1 packet and is waiting for feedback



rdt2.1: Summary of Sender Behaviour



Summary:

- New packets alternate between sequence numbers 0 and 1; retransmissions keep the same number
- Uses both **ACK**s and **NAK**s:
  - ACK → move on to next sequence
  - NAK (or corrupted feedback) → retransmit
- Handles corrupted data packets and ACKs/NAKs
- Stop-and-wait: waits for a valid ACK before sending new data



Discussion Question:

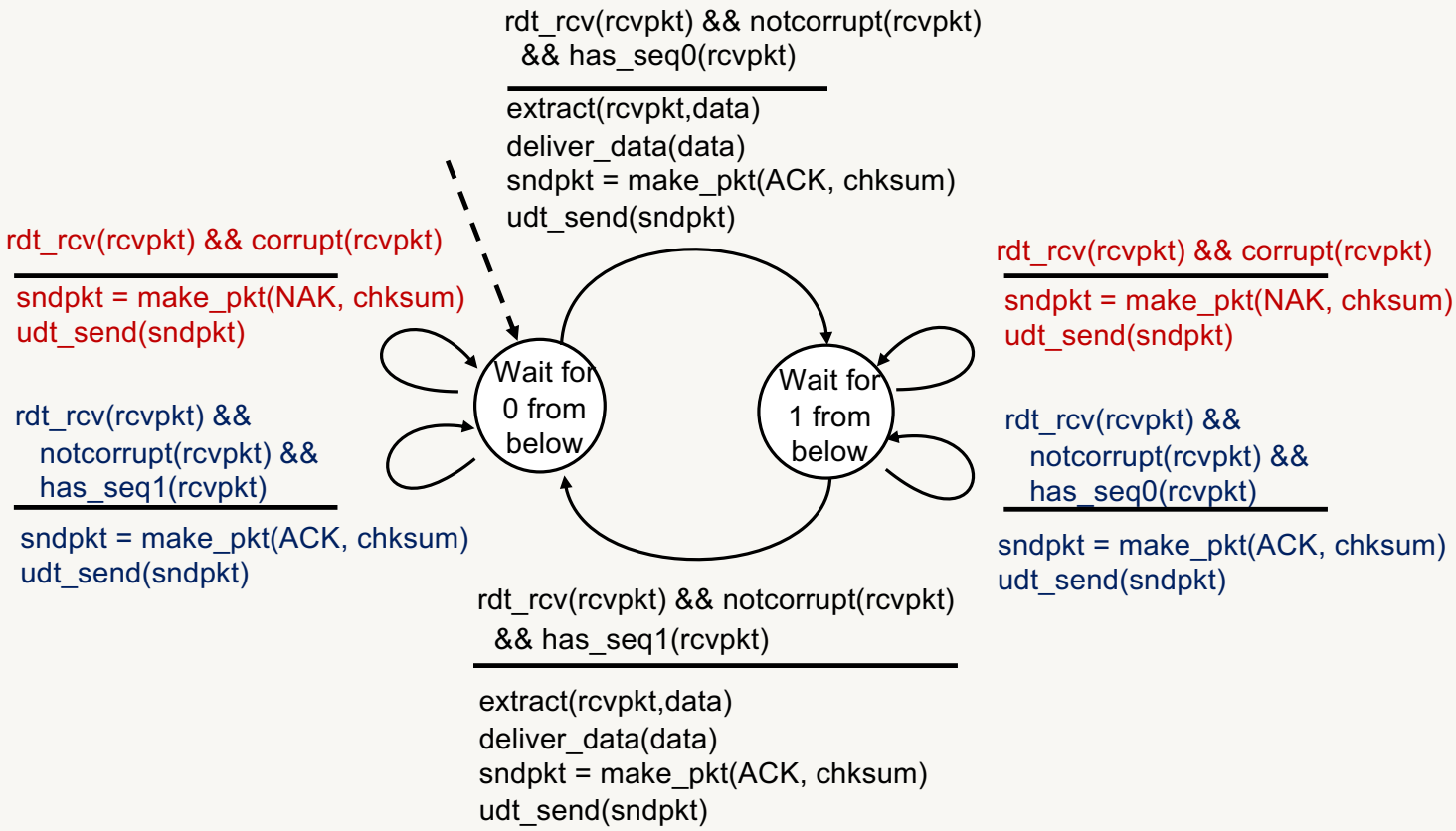
An rdt2.1 sender receives corrupted feedback for packet 0 and retransmits the same data with sequence number 1. Give a scenario in which this change causes duplicate delivery.

rdt2.1: Receiver States and Duplicate Detection



States

1. **Wait for 0 from below:** receiver expects the next packet to have sequence number 0
2. **Wait for 1 from below:** receiver expects the next packet to have sequence number 1



rdt2.1: Summary

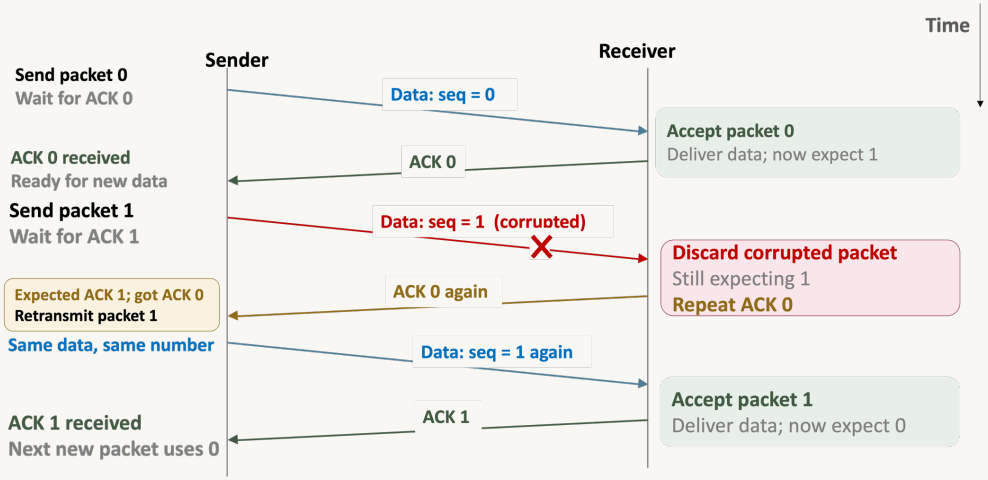


RDT / Protocol	Handles Bit Errors?	Handles Packet Loss?	Handles Corrupted ACK/NAK?	Handles Out-of-Order Packets?	Protocol Type
rdt1.0	Perfect channel → no errors	❌	N/A—no ACKs or NAKs used	❌	Simple transfer (no ACKs)
rdt2.0	✅	❌	❌ (does not handle corrupted ACK/NAK)	❌	Stop-and-Wait
rdt2.1	✅	❌	✅ (adds sequence numbers to handle corrupted ACK/NAK)	❌	Stop-and-Wait

rdt2.2: a NAK-Free Protocol



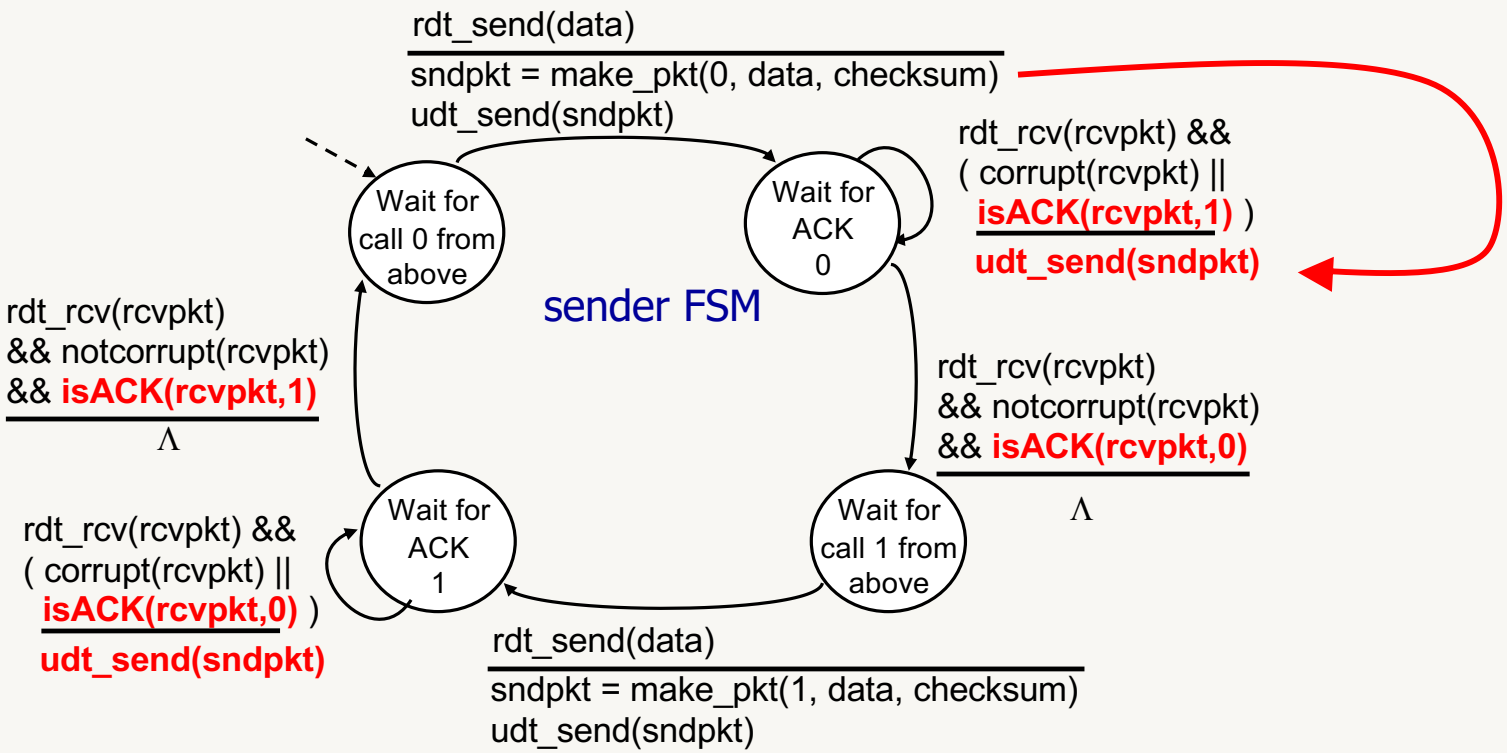
- **NAK-free alternative (rdt2.2)**
  - Replace NAK with **duplicate ACKs**:
    - Receiver sends ACK for the **last correctly received packet**
    - An ACK for the previous packet triggers retransmission of the current packet
- **Key change vs. rdt2.1**
  - ACKs now carry sequence numbers (0 or 1)
  - Sender checks ACK's sequence number to confirm which packet is being acknowledged



ACKs carry the sequence number of the last correctly received packet

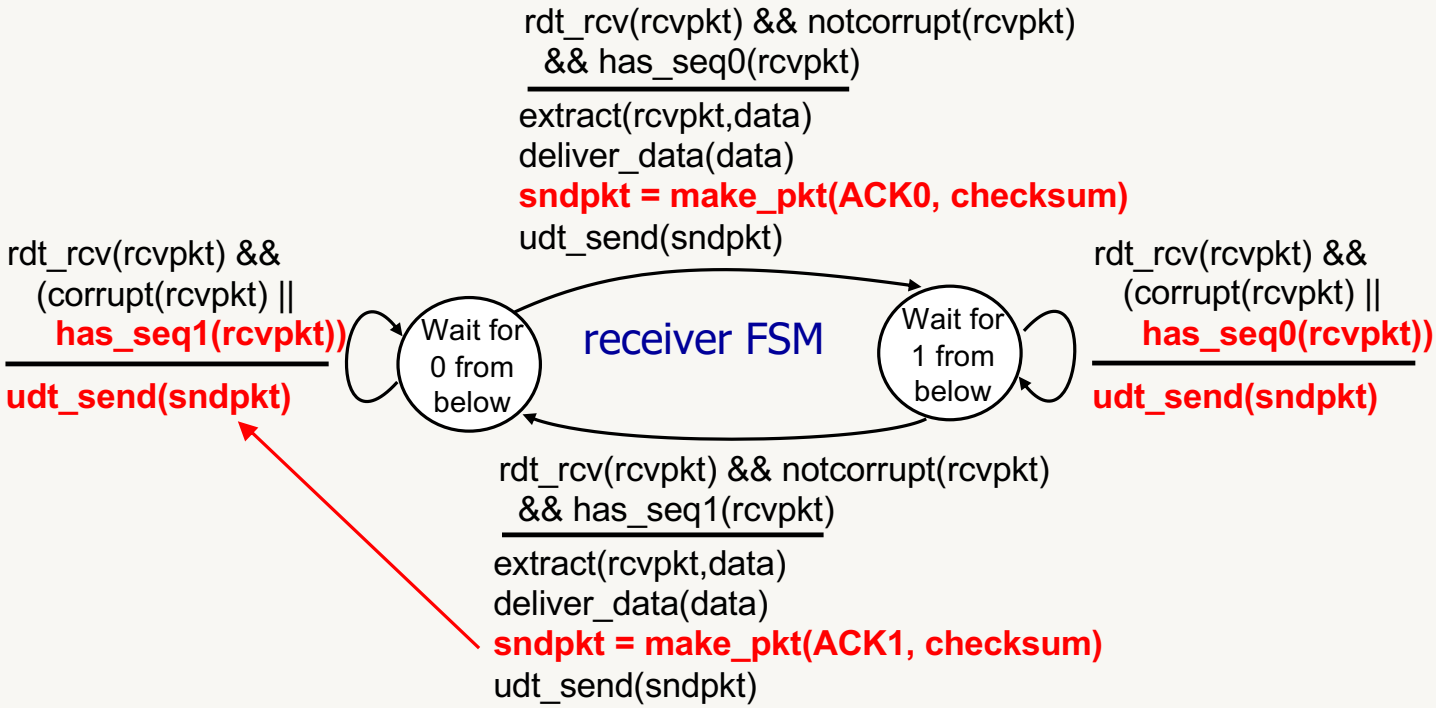
rdt2.2: Complete Sender FSM





rdt2.2: Complete Receiver FSM





rdt2.2: Key Features and Operation



Key Points of rdt2.2

- **No NAKs** - duplicates handled by sending repeated ACKs
- Sender retransmits if the ACK is corrupted or its number does not match the current packet
- Still a stop-and-wait protocol: the sender waits for a valid ACK for the current packet before sending new data
- Sequence numbers (0/1) prevent confusion between new and retransmitted packets

rdt2.2: Summary



RDT / Protocol	Handles Bit Errors?	Handles Packet Loss?	Handles Corrupted ACK/NAK?	Handles Out-of-Order Packets?	Protocol Type
rdt1.0	Perfect channel → no errors	❌	N/A—no ACKs or NAKs used	❌	Simple transfer (no ACKs)
rdt2.0	✅	❌	❌ (does not handle corrupted ACK/NAK)	❌	Stop-and-Wait
rdt2.1	✅	❌	✅ (adds sequence numbers to handle corrupted ACK/NAK)	❌	Stop-and-Wait
rdt2.2	✅	❌	✅ (uses duplicate ACKs instead of NAKs)	❌	Stop-and-Wait

rdt3.0: Lossy Channels with Bit Errors



- **New challenge**: Channel can now **lose packets** in addition to corrupting them
  - Protocol must handle:
    - **Detecting packet loss**
    - **Recovering from packet loss**
- **Handling packet loss**
  - Use existing mechanisms: checksums, sequence numbers, ACKs, retransmissions
  - Add new mechanism: **timeout-based retransmission**
- **Sender strategy**
  - If sender transmits data but receives no ACK (due to lost data or ACK), assume packet lost
  - After waiting a certain time, retransmit the packet
  - Time value chosen so loss is likely (not guaranteed)

rdt3.0: Lossy Channels with Bit Errors



- **Timer requirements**
  - **Start timer** each time a packet (new or retransmission) is sent
  - **Stop timer** once correct ACK is received
  - **Timeout interrupt** triggers retransmission if ACK not received



timeout

- **Issues with delay vs. loss**
  - A delayed packet may look "lost" → causes unnecessary retransmission
  - Protocol tolerates duplicates because **sequence numbers** distinguish new vs. old packets
- **Sender's viewpoint**
  - Doesn't matter if loss was data, ACK, or just delay → always retransmit
  - Retransmission is the universal recovery action
- **Protocol name**
  - With alternating sequence numbers (0,1), this is known as the **Alternating-Bit Protocol**

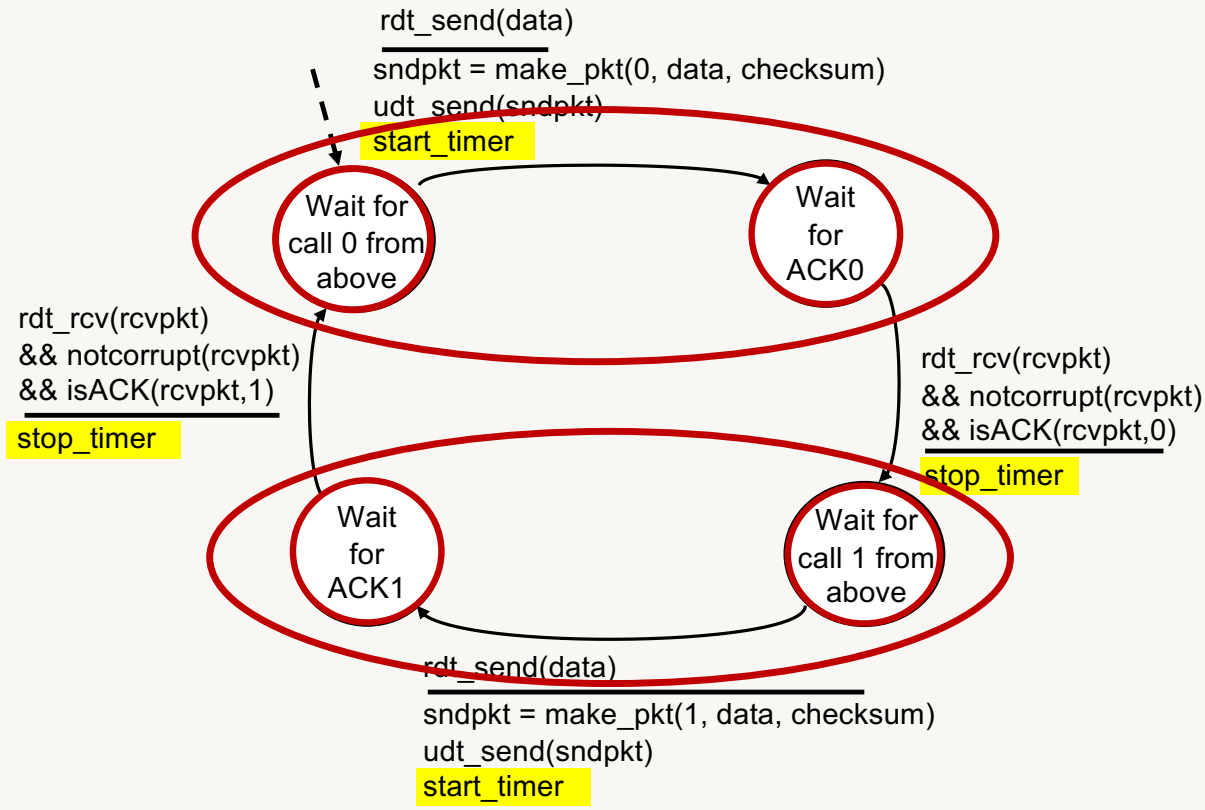
rdt3.0: Sender



Sender alternates between sequence numbers **0 and 1**

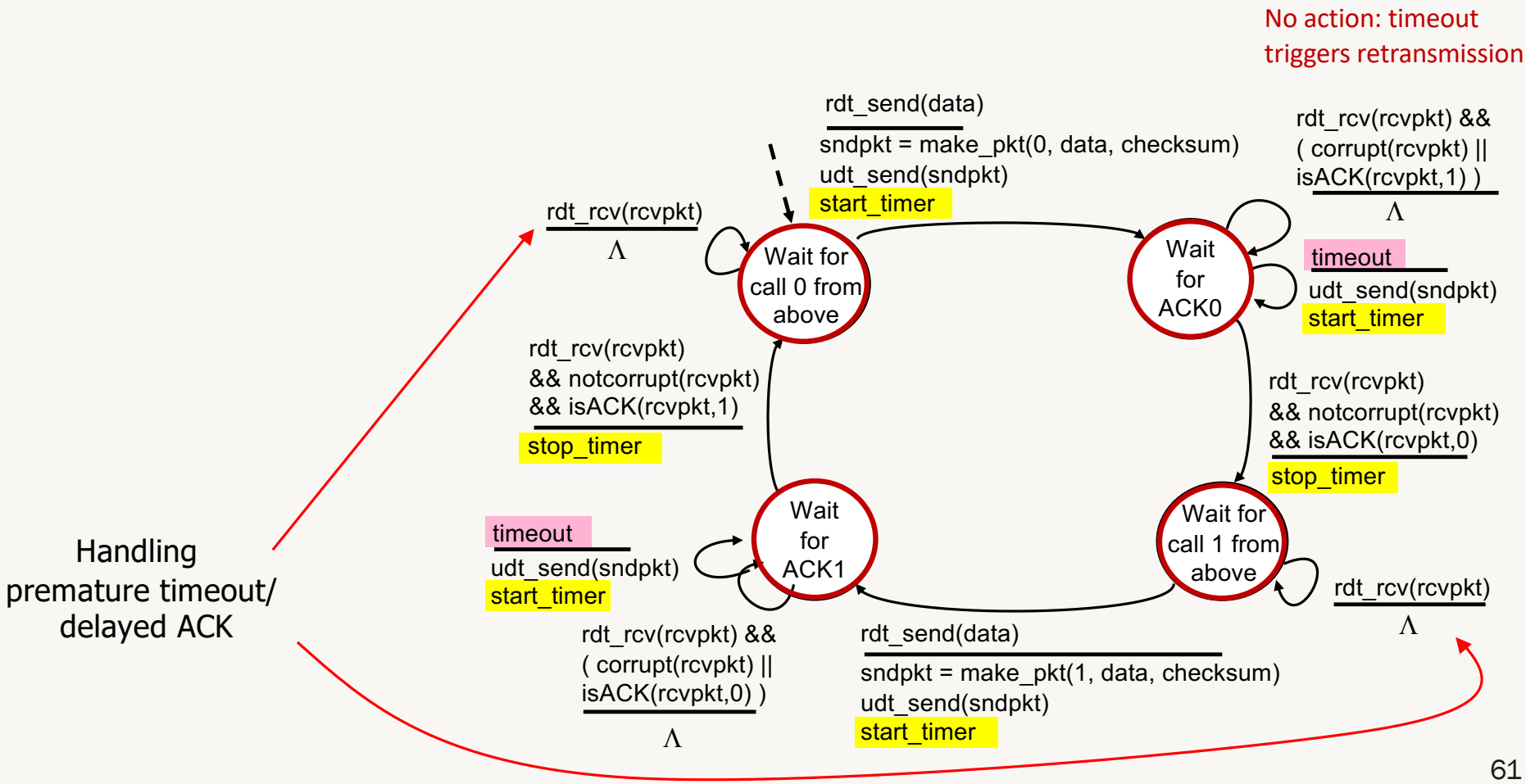
States:

1. **Wait for call 0 from above**
   - Ready to send new data packet with seq=0
2. **Wait for ACK 0**
   - Waiting for an ACK confirming seq=0
3. **Wait for call 1 from above**
   - Ready to send new data packet with seq=1
4. **Wait for ACK 1**
   - Waiting for an ACK confirming seq=1



rdt3.0: Sender





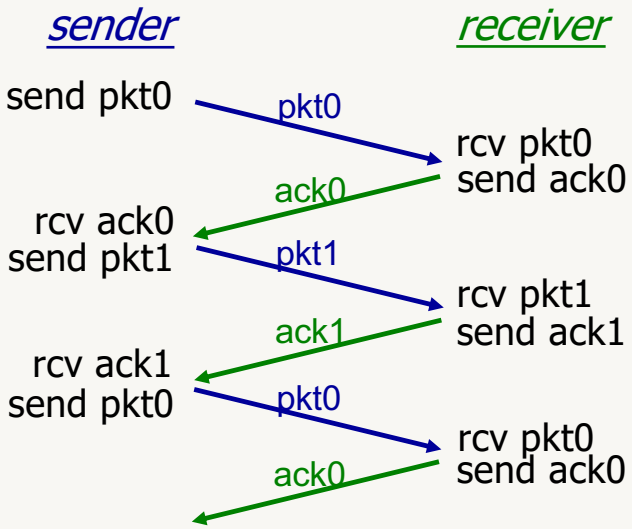
rdt3.0: Receiver



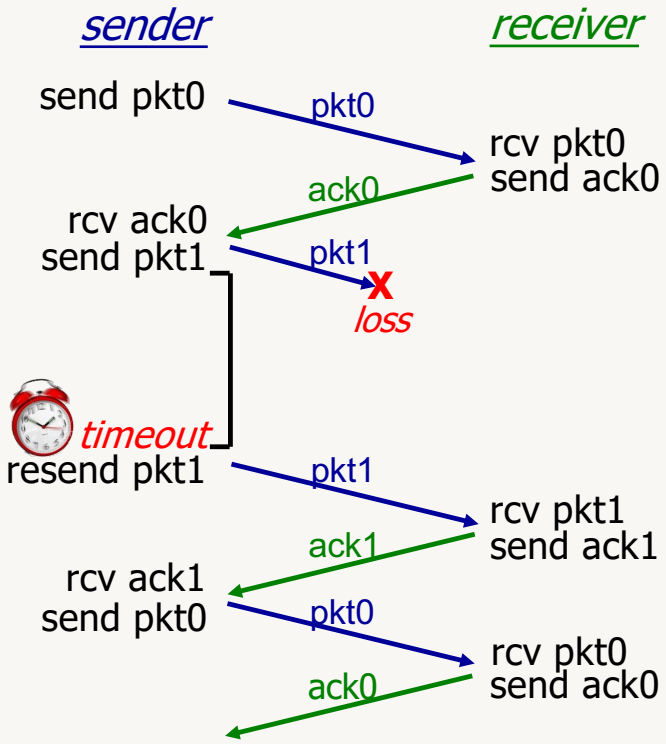
The rdt3.0 receiver uses the same FSM as rdt2.2

Operation of rdt3.0



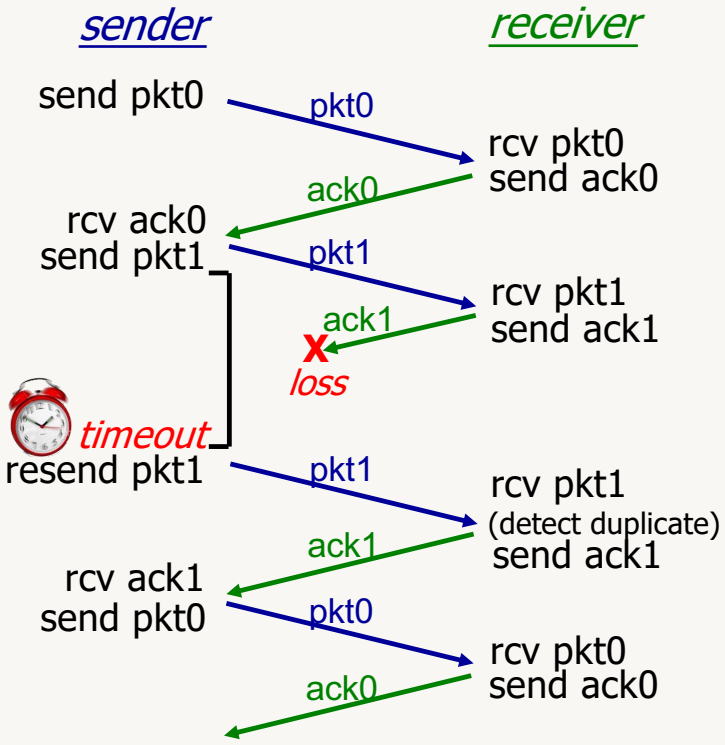


Operation with no lost or delayed packets

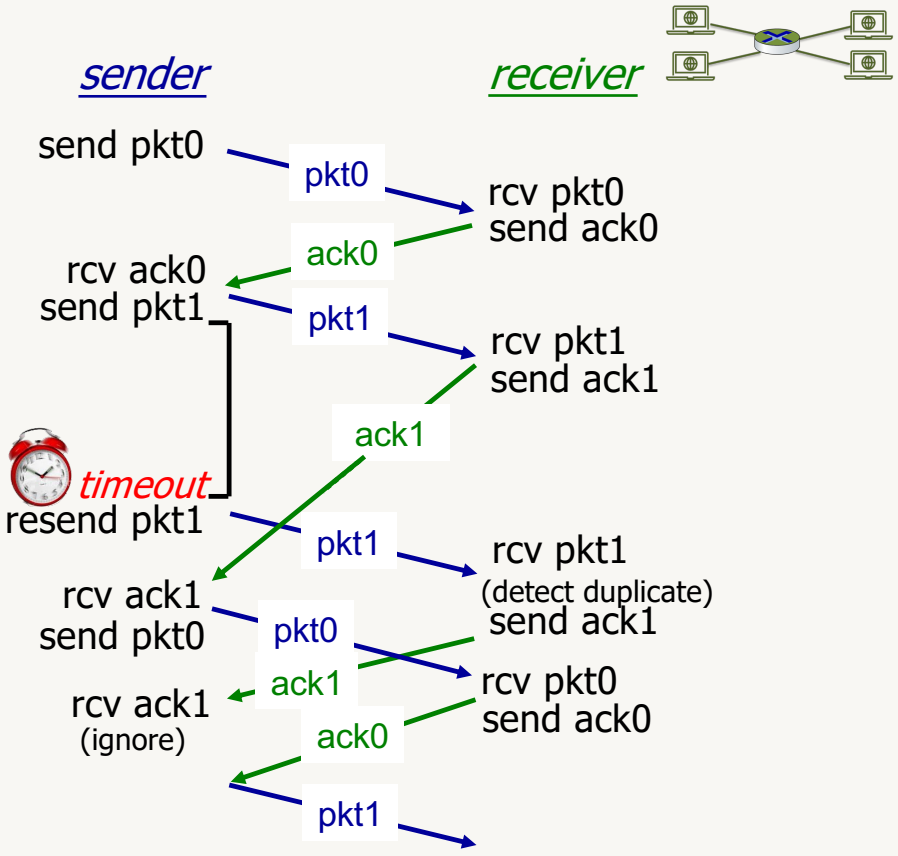


Operation when data packets are lost

Operation of rdt3.0



ACK loss

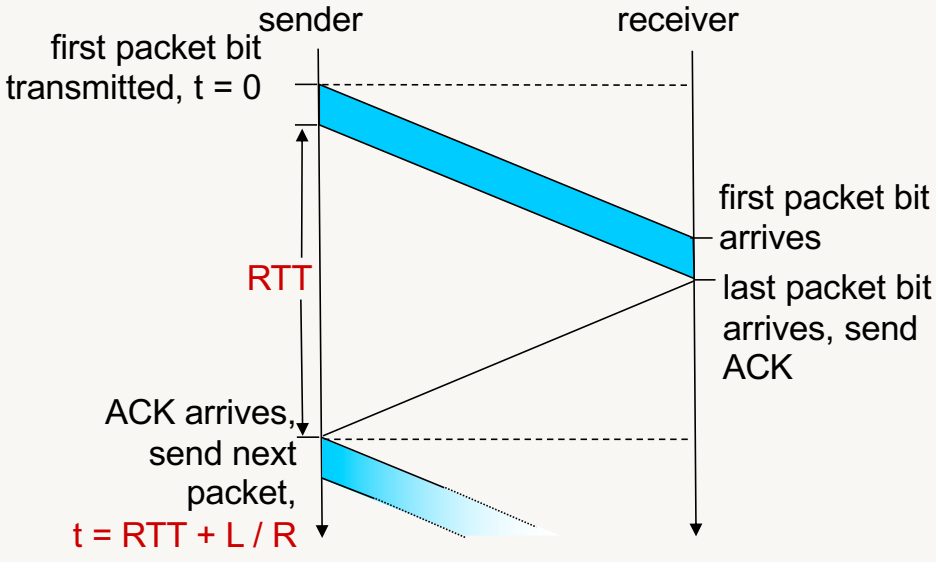


premature timeout/ delayed ACK

Performance of rdt3.0: stop-and-wait



- **Problem with rdt3.0 (stop-and-wait)**
  - Reliable, but inefficient on high-speed networks
  - Sends one packet, then waits for its ACK
- **Performance example: cross-country link**
  - RTT is much longer than packet transmission time
  - Sender is mostly idle, with very low utilization
  - Throughput is far below link capacity
  - Processing and queuing delays further reduce performance



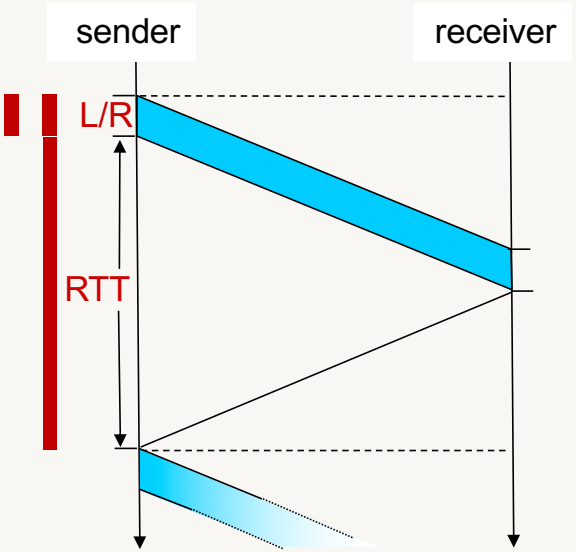
Performance of rdt3.0: stop-and-wait



$$U_{sender} = \frac{L / R}{RTT + L / R}$$

$$= \frac{.008}{30.008}$$

$$= 0.00027$$



- rdt3.0 suffers from poor performance
- the protocol itself bottlenecks the capacity of the channel

rdt3.0: stop-and-wait



RDT / Protocol	Handles Bit Errors?	Handles Packet Loss?	Handles Corrupted ACK/NAK?	Handles Out-of-Order Packets?	Protocol Type
rdt1.0	Perfect channel → no errors	❌	N/A—no ACKs or NAKs used	❌	Simple transfer (no ACKs)
rdt2.0	✅	❌	❌ (does not handle corrupted ACK/NAK)	❌	Stop-and-Wait
rdt2.1	✅	❌	✅ (adds sequence numbers to handle corrupted ACK/NAK)	❌	Stop-and-Wait
rdt2.2	✅	❌	✅ (uses duplicate ACKs instead of NAKs)	❌	Stop-and-Wait
rdt3.0	✅	✅	✅	❌	Stop-and-Wait

Pipelined protocols



Solution: Pipelining

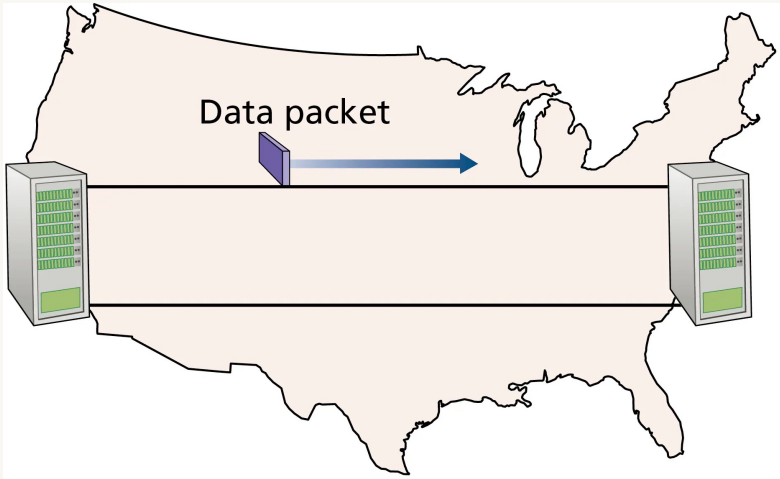
- Sender can transmit **multiple packets before waiting for ACKs**
- This significantly increases utilization (e.g., sending 3 packets triples efficiency)
- Packets in transit form a **pipeline**

Consequences of pipelining:

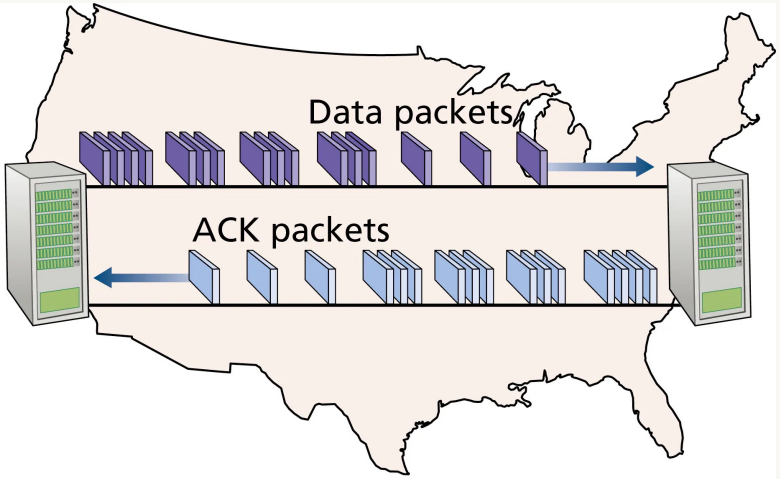
- **Larger sequence number range** → each unacknowledged packet must have a unique sequence number
- **Sender buffering** → store unacknowledged packets for possible retransmission
- **Receiver buffering** (in some cases) → hold out-of-order but correctly received packets
- Error recovery strategies:

Pipelined protocols





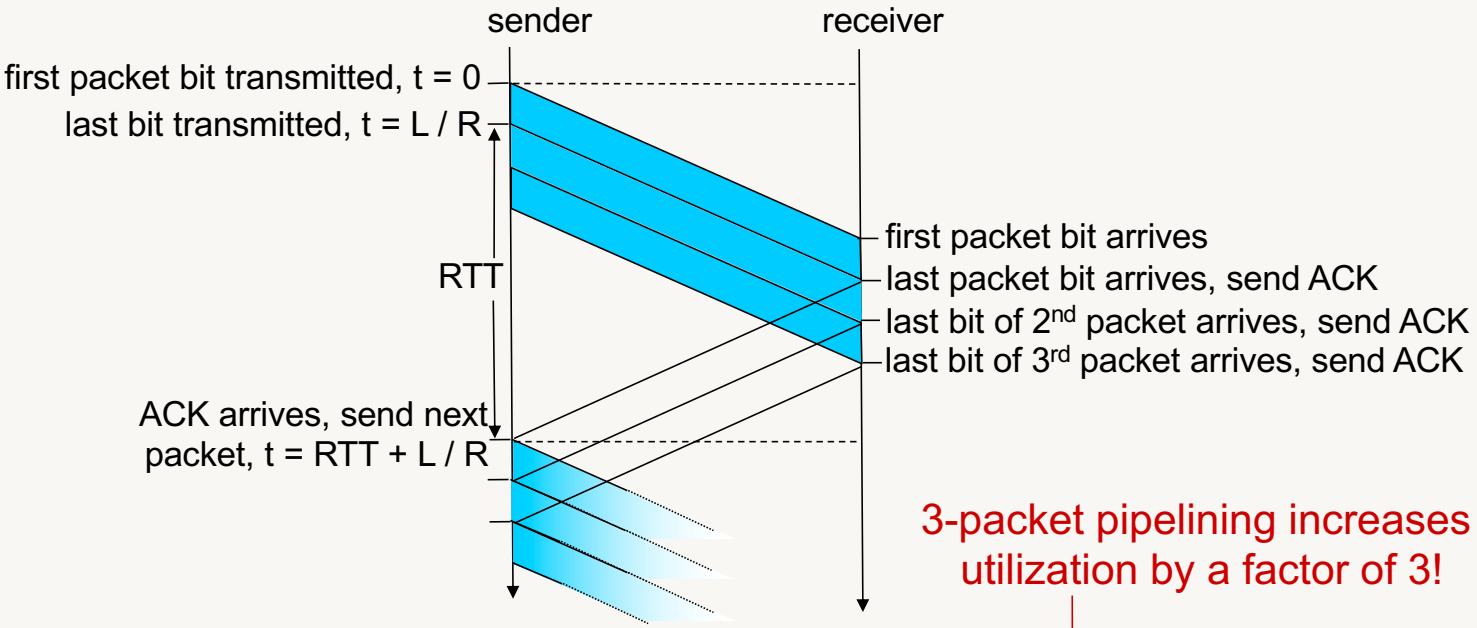
A stop-and-wait protocol in operation



A pipelined protocol in operation

Pipelined protocols





3-packet pipelining increases utilization by a factor of 3!

$$U_{sender} = \frac{3L / R}{RTT + L / R} = \frac{.0024}{30.008} = 0.00081$$

Go-Back-N

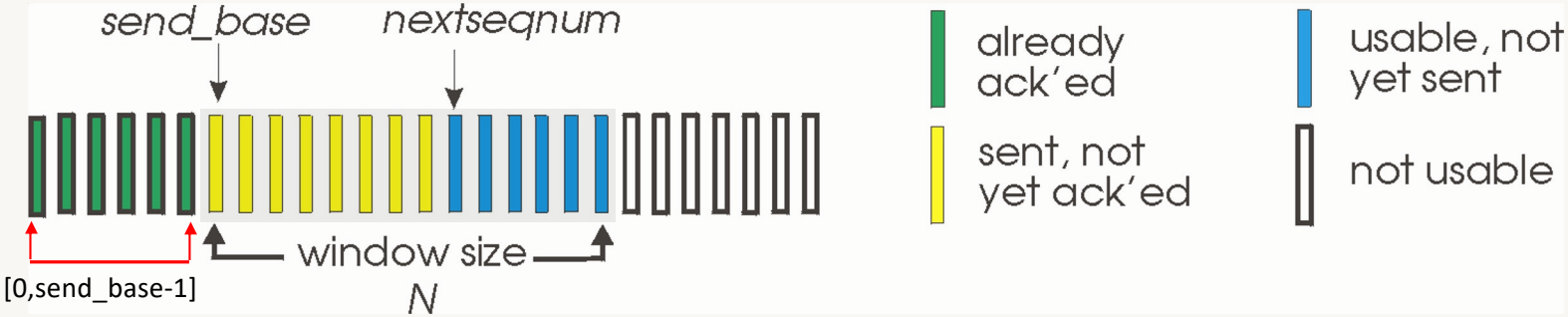


- **Basic idea**:
  - Sender can send multiple packets without waiting for ACKs
  - Maximum of **N unacknowledged packets** allowed in the pipeline (window size)
  - Operates as a **sliding window protocol**
- **Sequence number handling**:
  - Packets are labeled with sequence numbers
  - Window covers range [send_base, nextseqnum - 1]
  - As ACKs are received, the window slides forward
  - With *k* bits for sequence numbers → range = 0 … 2^k – 1

Go-Back-N: Sender



- Sender keeps a sliding window of up to N unacknowledged packets, each labeled with a *k*-bit sequence number in the header



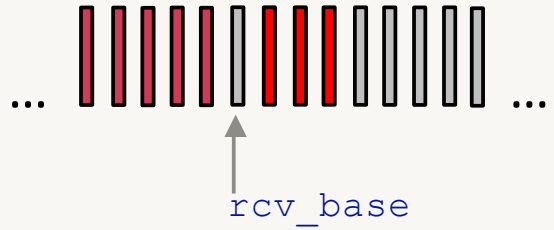
- Cumulative ACK(n): confirms all packets ≤ n; window slides to n+1 (n is seq#)
- Sender uses one timer for the oldest unACKed packet
- timeout(n): retransmits packet n and all later packets in the window

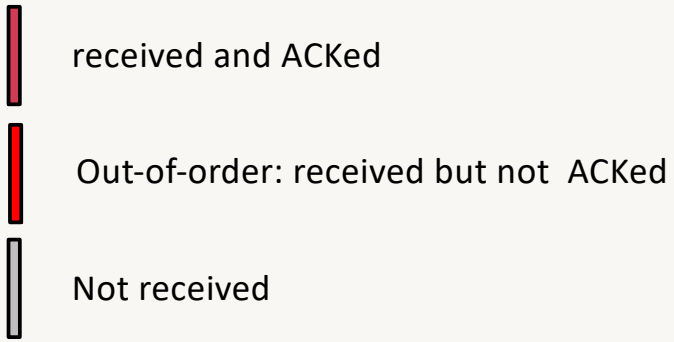
Go-Back-N: Receiver



- Receiver sends ACK for highest in-order seq# received (may duplicate ACKs)
- Tracks only rcv_base
- Out-of-order packets: either discard or buffer (design choice), then re-ACK last in-order packet

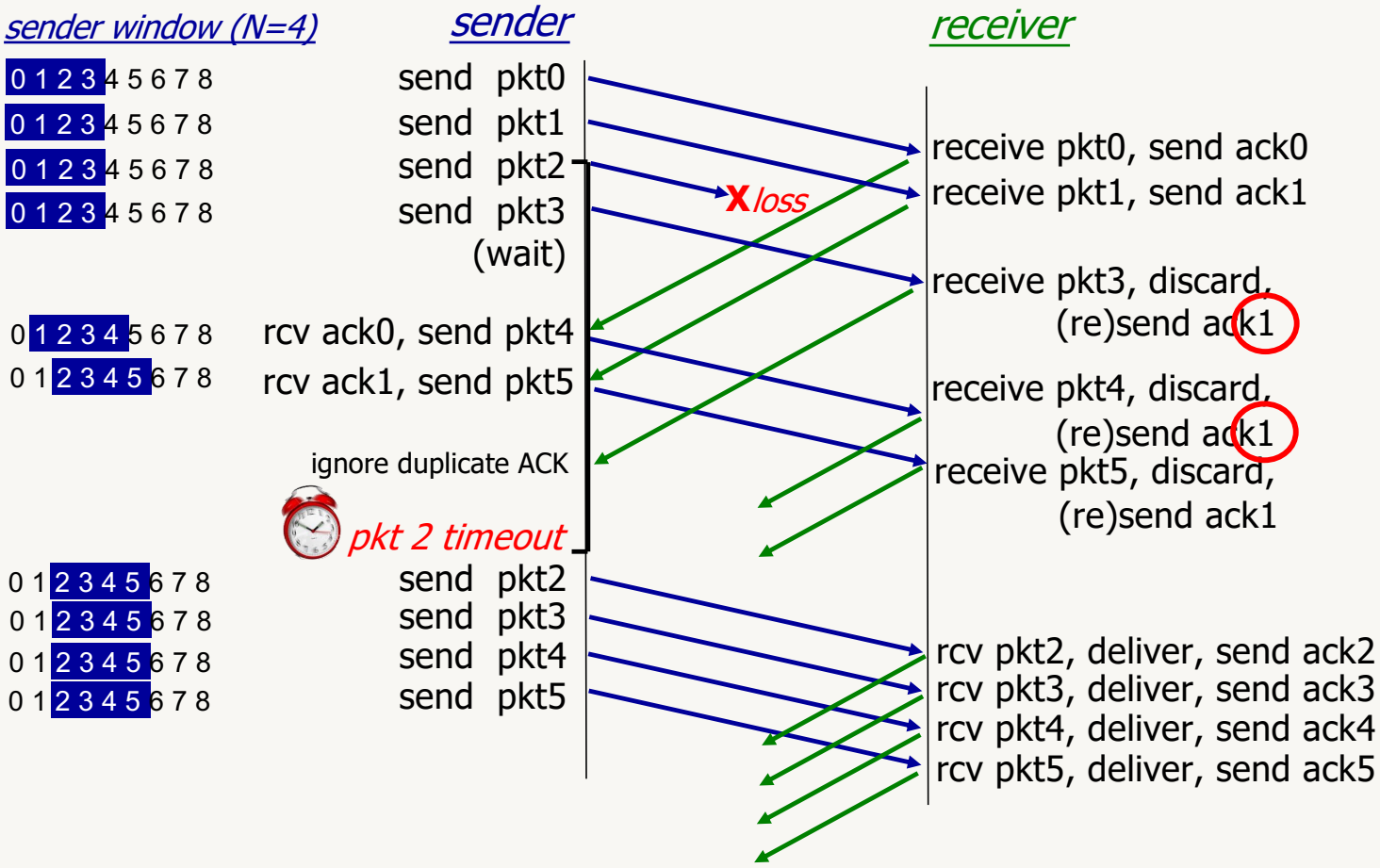
Receiver view of sequence number space:





Operation of Go-Back-N





Go-Back-N



RDT / Protocol	Handles Bit Errors?	Handles Packet Loss?	Handles Corrupted ACK/NAK?	Handles Out-of-Order Packets?	Protocol Type
rdt1.0	Perfect channel → no errors	❌	N/A—no ACKs or NAKs used	❌	Simple transfer (no ACKs)
rdt2.0	✅	❌	❌ (does not handle corrupted ACK/NAK)	❌	Stop-and-Wait
rdt2.1	✅	❌	✅ (adds sequence numbers to handle corrupted ACK/NAK)	❌	Stop-and-Wait
rdt2.2	✅	❌	✅ (uses duplicate ACKs instead of NAKs)	❌	Stop-and-Wait
rdt3.0	✅	✅	✅	❌	Stop-and-Wait
Go-Back-N (GBN)	✅	✅	✅ (duplicate ACKs handle corruption)	❌ (discards out-of-order packets)	**Pipelined**

Selective Repeat



- **GBN inefficiency**
  - Sender can keep the pipeline full
  - But a **single packet error** forces retransmission of many packets, even if correctly received but out-of-order
  - High error rates → pipeline clogged with unnecessary retransmissions
- **Selective Repeat (SR) improvement**
  - Retransmit only the lost or **corrupted packets**
  - Each correctly received packet is **individually acknowledged**
  - Window size = **N**, but sender may already have ACKs for some packets within the window
  - Receiver can buffer out-of-order packets until missing ones arrive, then deliver in-order to the upper layer

Selective Repeat



Selective Repeat (SR) sender actions:

- **Data received from above**
  - Sender checks if sequence number is within its window
  - If yes → data is packetized and sent
  - If not → data is buffered or returned to upper layer (like in GBN)
- **Timeout**
  - Each packet has its own logical timer (only that packet is retransmitted on timeout)
  - A single hardware timer can simulate multiple logical timers
- **ACK received**
  - Sender marks the acknowledged packet as received (if in window)
  - If ACK is for the **base packet (send_base)**, window moves forward to the next unacknowledged packet
  - Newly included packets in the window are transmitted if data is available

Selective Repeat

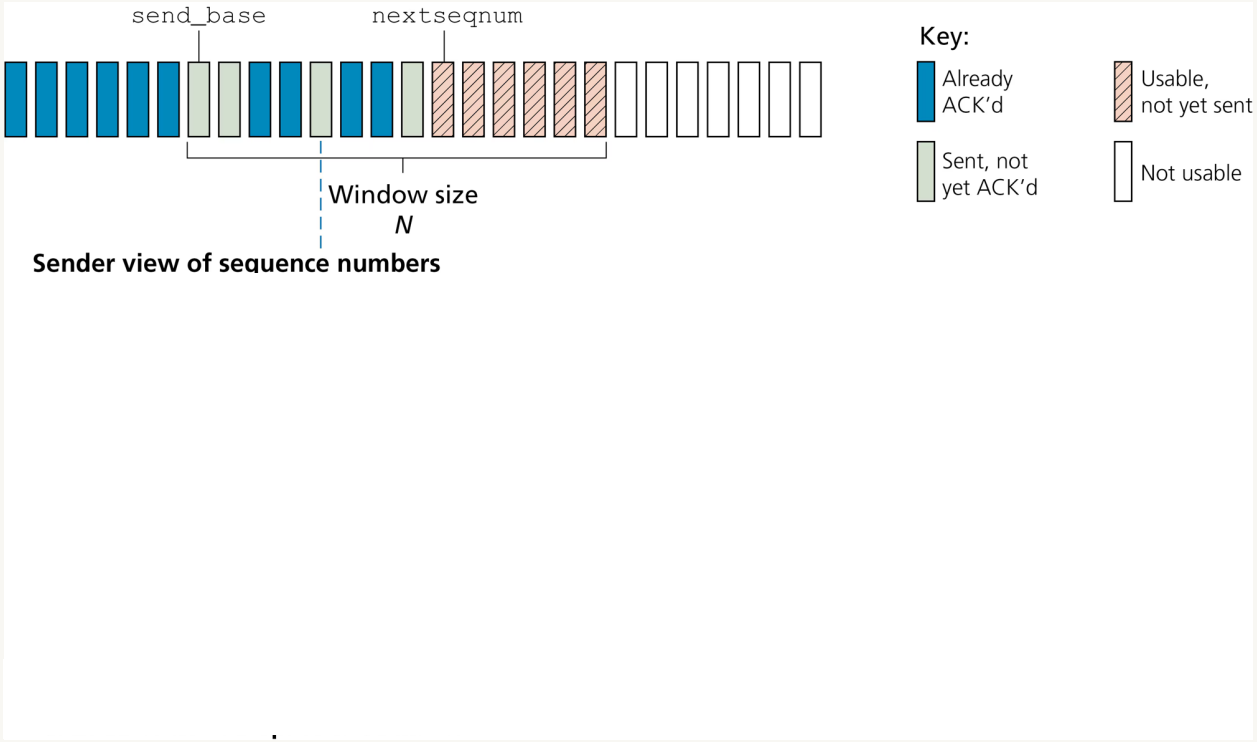


Selective Repeat (SR) receiver actions:

- **Case 1: Packet in receiver window [rcv_base,rcv_base+N−1]**
  - Send selective ACK to sender
  - If new (not received before) → buffer it
  - If it matches rcv_base → deliver it and all consecutively buffered packets to the upper layer
  - Slide the receive window forward by number of packets delivered
- **Case 2: Packet already received (sequence number in [rcv_base−N,rcv_base−1]**
  - Send ACK again, even if it was already acknowledged (re-ACK)
- **Case 3: Otherwise**
  - Ignore the packet

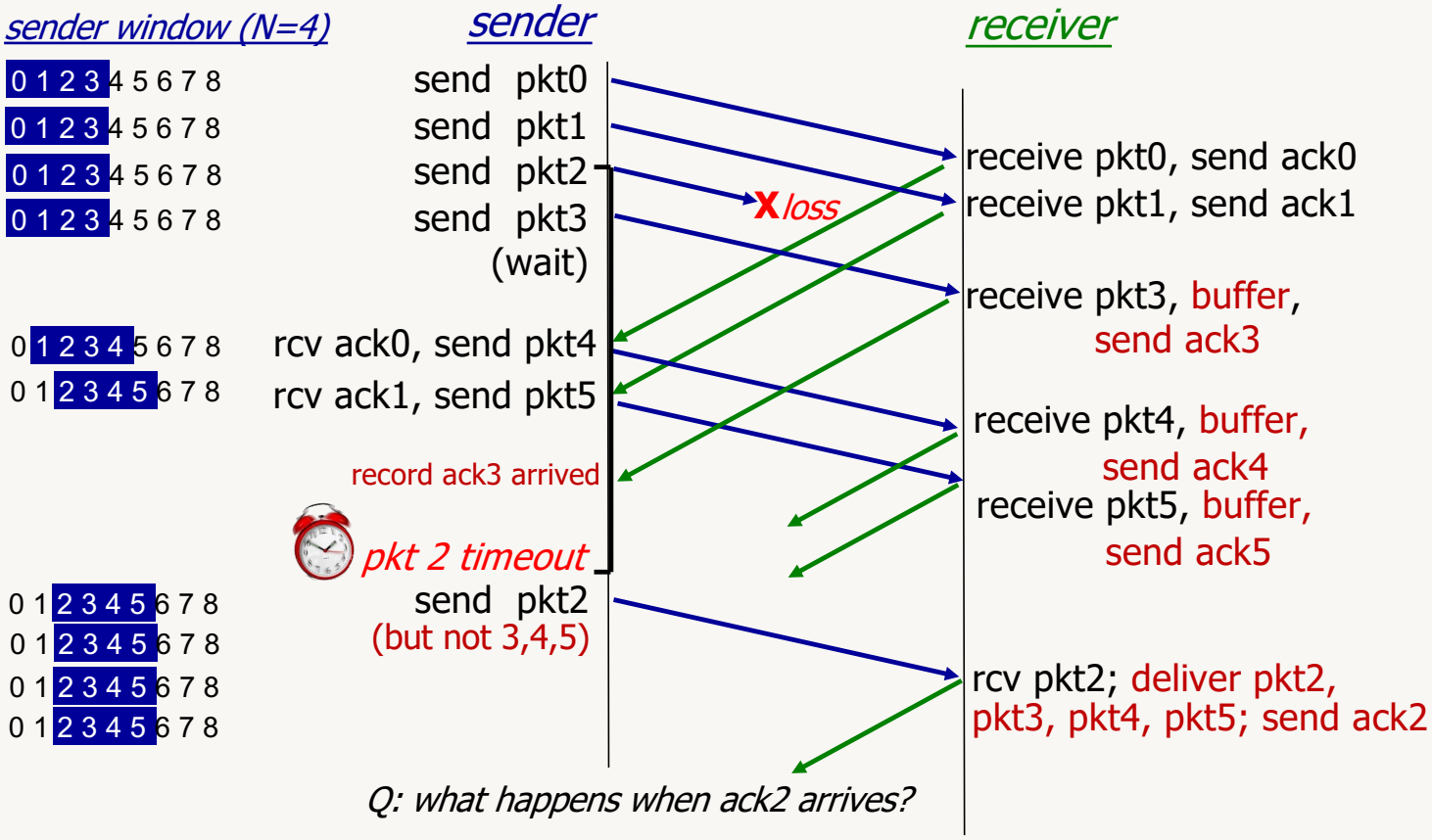
Selective Repeat: Sender, Receiver windows





Operation of Selective Repeat



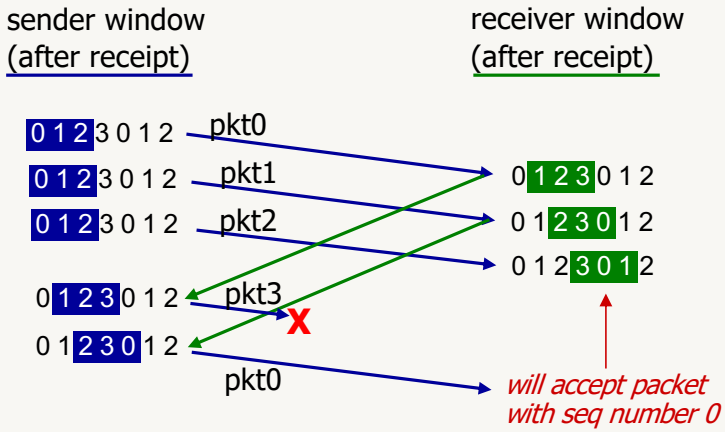


Problem with Selective Repeat

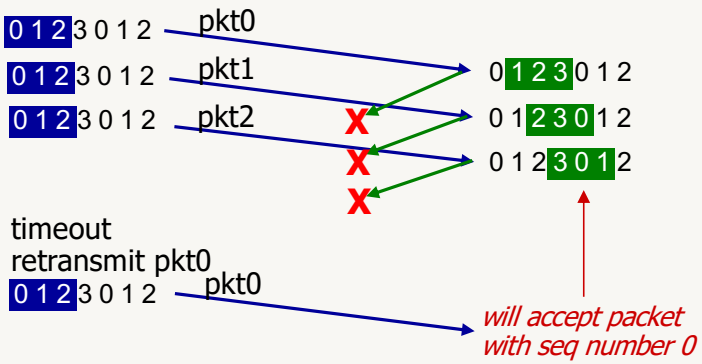


example:

- seq #s: 0, 1, 2, 3 (base 4 counting)
- window size=3



no problem



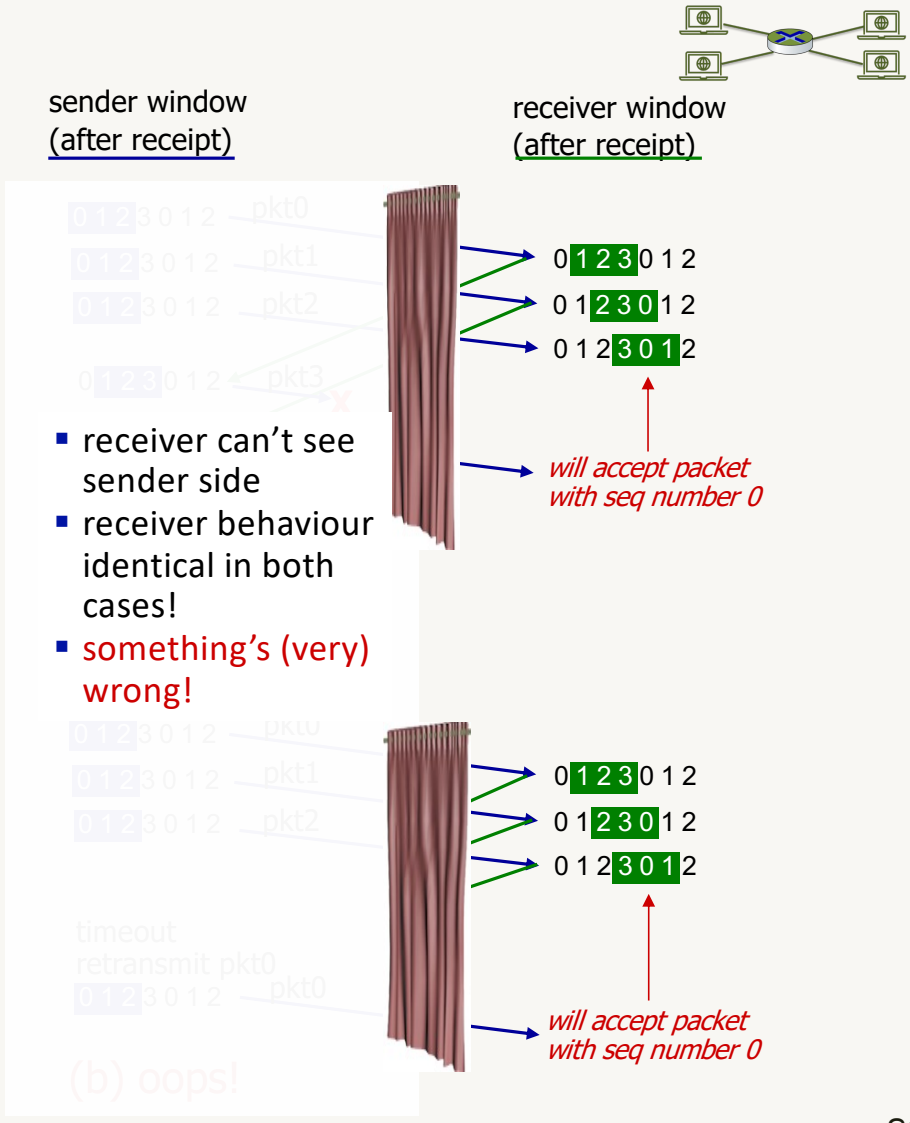
oops!

Problem with Selective Repeat

example:

- seq #s: 0, 1, 2, 3 (base 4 counting)
- window size=3

Q: what relationship is needed between sequence # size and window size to avoid problem in scenario (b)?



Summary



RDT / Protocol	Handles Bit Errors?	Handles Packet Loss?	Handles Corrupted ACK/NAK?	Handles Out-of-Order Packets?	Protocol Type
rdt1.0	Perfect channel → no errors	❌	N/A—no ACKs or NAKs used	❌	Stop-and-Wait
rdt2.0	✅	❌	❌ (does not handle corrupted ACK/NAK)	❌	Stop-and-Wait
rdt2.1	✅	❌	✅ (adds sequence numbers to handle corrupted ACK/NAK)	❌	Stop-and-Wait
rdt2.2	✅	❌	✅ (uses duplicate ACKs instead of NAKs)	❌	Stop-and-Wait
rdt3.0	✅	✅	✅	❌	Stop-and-Wait
Go-Back-N (GBN)	✅	✅	✅ (duplicate ACKs handle corruption)	❌ (discards out-of-order packets)	**Pipelined**
Selective Repeat (SR)	✅	✅	✅	✅ (buffers out-of-order packets)	**Pipelined**

References



- Ch. 3, Kurose and Ross



Questions?